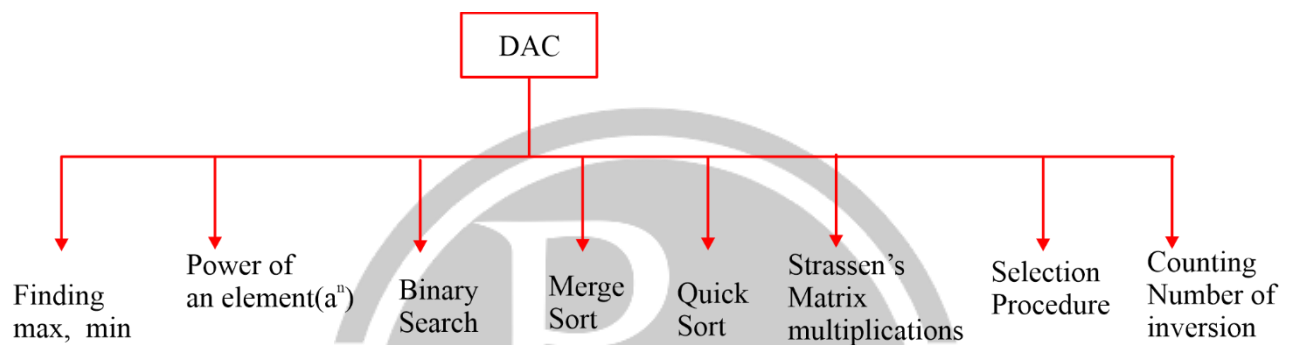


2

DIVIDE AND CONQUER

2.1 DAC Application



2.2 Finding Maximum Minimum element

Recurrence Relation:

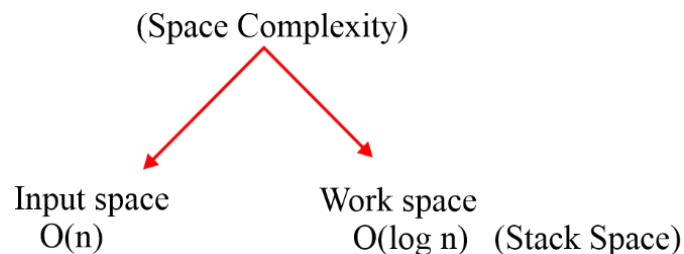
$$T(n) = \begin{cases} 1 & \text{if } n=1 \text{ or } n=2 \\ 2T\left(\frac{n}{2}\right) + 1; & n > 2 \end{cases}$$

Time Complexity:

$$T(n) = O(n)$$

- Time complexity is same for every case (Best case/Worst case).

Space Complexity:





$$\begin{aligned}\text{Space Complexity} &= O(n) + O(\log n) \\ &= O(n)\end{aligned}$$

Number of comparisons to find maximum / minimum element on an given array of n elements:

$$\text{Comparison} = \frac{3n}{2} - 2$$

2.3 Power of an Element

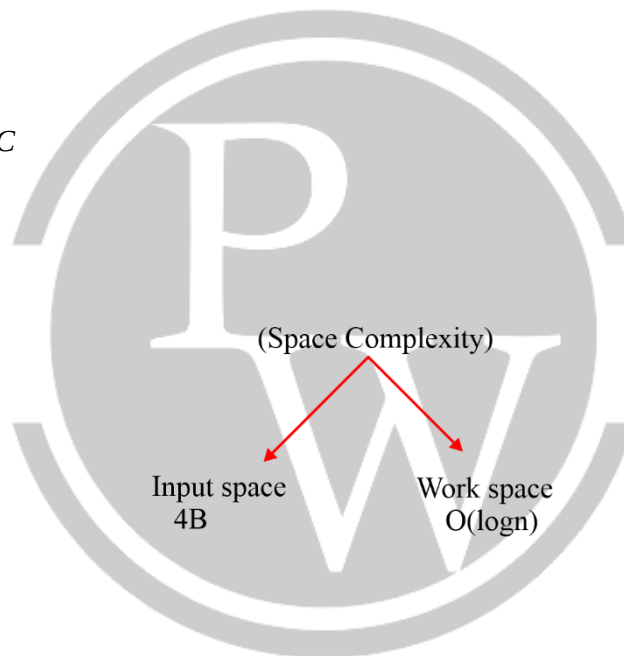
Recurrence relation:

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ T\left(\frac{n}{2}\right) + 1; & n > 1 \end{cases}$$

Time Complexity:

$$\begin{aligned}T(n) &= T\left(\frac{n}{2}\right) + C \\ T(n) &= O(\log n)\end{aligned}$$

Space Complexity:



$$\begin{aligned}\text{Space Complexity} &= 4B + O(\log n) \\ &= O(\log n)\end{aligned}$$

Number of multiplications to find a^n

$$\text{Multiplication} = O(\log n)$$

2.4 Binary Search

Given a sorted array and an element x, need to return the index of element x if it is present then 1, otherwise – 1.

Recurrence relation:

$$T(n) = \begin{cases} 1; & n=1 \\ T\left(\frac{n}{2}\right) + C; & n > 1 \end{cases}$$